

- Description

This module manages a smart solar panel system, ensuring that:

1. **The power output of solar panels is continuously monitored to optimize energy generation.**
2. **If power generation drops below an efficiency threshold, the panel tilt angle is adjusted for better sunlight exposure.**
3. **If the power output reaches the optimal range, the tilt remains unchanged to avoid unnecessary movement.**
4. **A log of all power readings, tilt adjustments, and efficiency improvements is maintained for energy optimization.**

This system helps maximize solar energy utilization, improve efficiency, and reduce maintenance costs.

- Specification

module SmartSolarPanelControlEx

var currentPowerOutput, maxOptimalPower, minOptimalPower: real

var panelTiltAngle: real

process updatePowerOutput(changeAmount: real)

functional scenario form:

currentPowerOutput >= minOptimalPower && Power output maintained optimally.

||

currentPowerOutput < minOptimalPower && panelTiltAngle = panelTiltAngle + 5.0, Panel tilt increased to enhance sunlight absorption.

||

currentPowerOutput > maxOptimalPower && panelTiltAngle = panelTiltAngle - 5.0, Panel tilt reduced to prevent overexposure.

end process

end module

- Java Code

```
package code;
import javax.swing.*;
import javax.swing.border.EmptyBorder;
import javax.swing.table.DefaultTableModel;
import java.awt.*;
import java.awt.event.*;
import java.io.*;
import java.nio.charset.StandardCharsets;
import java.text.DecimalFormat;
import java.time.LocalDateTime;
import java.time.format.DateTimeFormatter;
import java.util.*;
```

```

import java.util.List;
import java.util.concurrent.ConcurrentLinkedQueue;
import java.util.concurrent.atomic.AtomicBoolean;
/**
 * =====
 * c129_SmartSolarSystemEx
 * Domain: Smart solar panel with tilt control and power optimization
 *
 * Goals:
 * - Preserve original decision logic INSIDE ONE CORE FUNCTION:
 *   public synchronized void updatePowerOutput(double changeAmount)
 *   where we:
 *   (1) update currentPowerOutput by changeAmount,
 *   (2) adjust tilt angle based on min/max optimal power thresholds
 *
 *   (3) log & sample sensors & notify listeners.
 *
 * Features:
 * - Core SmartSolarPanel logic (kept in single core function for coverage)
 * - Sensors: irradiance, temperature, efficiency, DC bus voltage/current
 * - Controller: TiltController (physical limits & rate), MPPT estimator
 * - KPI: energy accumulation, PR (performance ratio) approximation
 * - Alert system & file-backed logger
 * - Swing GUI: live status, thresholds, operations, simulation panel, alerts, log tail
 * - Simulation engine: sun path/irradiance, cloud cover, ambient temp/wind
 * - Replay engine: parse sim steps from log and replay
 * - CLI harness: VT/FT/Z3/UVT test groups (comment-to-toggle single group)
 *
 * Compile: javac code/c129_SmartSolarSystemEx.java
 * Run GUI: java code.c129_SmartSolarSystemEx
 * Run CLI: java code.c129_SmartSolarSystemEx --cli
 * =====
 */
/* ===== Utilities ===== */
class SStr {
    static String fmt2(double v) { return new DecimalFormat("0.00").format(v); }

```

```

        static String fmt3(double v) { return new DecimalFormat("0.000").format(v); }
        static String now() { return LocalDateTime.now().format(DateTimeFormatter.ofPattern("yyyy-MM-dd HH:mm:ss")); }
    }
    /* ===== Logger ===== */
    class SDataLogger {
        private final File file;
        private final ConcurrentLinkedQueue<String> inMemory = new ConcurrentLinkedQueue<>();
        private final int cap;
        SDataLogger(String path, int inMemoryCap) {
            this.file = new File(path);
            this.cap = Math.max(200, inMemoryCap);
            try {
                if (!file.exists()) {
                    File parent = file.getParentFile();
                    if (parent != null) parent.mkdirs();
                    file.createNewFile();
                }
            } catch (IOException e) {
                System.err.println("[Logger] Create file failed: " + e.getMessage());
            }
        }
        synchronized void log(String message) {
            String line = "[" + SStr.now() + "] " + message;
            inMemory.add(line);
            while (inMemory.size() > cap) inMemory.poll();
            try (FileOutputStream fos = new FileOutputStream(file, true);
                OutputStreamWriter osw = new OutputStreamWriter(fos, StandardCharsets.UTF_8);
                BufferedWriter bw = new BufferedWriter(osw)) {
                bw.write(line);
                bw.newLine();
            } catch (IOException e) {
                System.err.println("[Logger] Write failed: " + e.getMessage());
            }
        }
        List<String> recent(int n) {
            ArrayList<String> list = new ArrayList<>(inMemory);
            if (list.size() <= n) return list;
            return list.subList(list.size() - n, list.size());
        }
    }

```

```

    }
    File getFile() { return file; }
}
/* ===== Alerts ===== */
class SAlertSystem {
    enum Severity { INFO, WARNING, CRITICAL }
    static class Alert {
        final Severity severity;
        final String message;
        final LocalDateTime time = LocalDateTime.now();
        Alert(Severity s, String m) { this.severity = s; this.message =
m; }
        @Override public String toString() { return "[" + time + "]" +
severity + " " + message; }
    }
    private final List<Alert> alerts = new ArrayList<>();
    private final SDataLogger logger;
    SAlertSystem(SDataLogger logger) { this.logger = logger; }
    void push(Severity s, String msg) {
        Alert a = new Alert(s, msg);
        alerts.add(a);
        logger.log("ALERT " + s + " :: " + msg);
        System.out.println(a);
    }
    List<Alert> recent(int n) {
        if (alerts.size() <= n) return new ArrayList<>(alerts);
        return new ArrayList<>(alerts.subList(alerts.size() - n, alerts
.size()));
    }
}
/* ===== Sensors ===== */
abstract class SSensor {
    protected String name;
    protected double value;
    protected boolean ok = true;
    protected String status = "OK";
    protected Random rnd = new Random();
    SSensor(String name) { this.name = name; }
    abstract void update(SolarContext ctx);
    String getName() { return name; }
    double getValue() { return value; }
    boolean isOk() { return ok; }
    String getStatus() { return status; }
}

```

```

class SolarContext {
    public double currentPowerOutput; // W
    public double minOptimalPower;    // W
    public double maxOptimalPower;    // W
    public double panelTiltAngle;     // degrees
    public double irradiance;         // W/m^2
    public double cellTemperature;    // °C
    public double ambientTemperature; // °C
    public double windSpeed;          // m/s
    public double dcVoltage;          // V
    public double dcCurrent;          // A
    public boolean trackingEnabled;    // for controller info
    public double cumulativeEnergyWh; // Wh

    SolarContext copy() {
        SolarContext c = new SolarContext();
        c.currentPowerOutput = currentPowerOutput;
        c.minOptimalPower    = minOptimalPower;
        c.maxOptimalPower    = maxOptimalPower;
        c.panelTiltAngle     = panelTiltAngle;
        c.irradiance         = irradiance;
        c.cellTemperature    = cellTemperature;
        c.ambientTemperature = ambientTemperature;
        c.windSpeed          = windSpeed;
        c.dcVoltage          = dcVoltage;
        c.dcCurrent          = dcCurrent;
        c.trackingEnabled    = trackingEnabled;
        c.cumulativeEnergyWh = cumulativeEnergyWh;
        return c;
    }
}

class IrradianceSensor extends SSensor {
    IrradianceSensor() { super("IrradianceSensor"); }
    @Override
    void update(SolarContext ctx) {
        value = Math.max(0, ctx.irradiance + (rnd.nextDouble() - 0.5) *
20.0);
        ok = (value >= 0 && value <= 1400);
        status = ok ? (value < 100 ? "LOW" : (value < 800 ? "MEDIUM" :
"HIGH")) : "OUT_OF_RANGE";
    }
}

class CellTempSensor extends SSensor {
    CellTempSensor() { super("CellTempSensor"); }
    @Override

```

```

    void update(SolarContext ctx) {
        value = ctx.cellTemperature + (rnd.nextDouble() - 0.5) * 0.8;
        ok = (value > -20 && value < 110);
        status = ok ? (value > 80 ? "HOT" : "OK") : "OUT_OF_RANGE";
    }
}

class EfficiencySensor extends SSensor {
    EfficiencySensor() { super("EfficiencySensor"); }
    @Override
    void update(SolarContext ctx) {
        // naive efficiency estimate from power = V*I and irradiance ar
        ea factor (assume 1.6 m^2)
        double panelArea = 1.6; // m^2
        double vin = Math.max(1e-
3, ctx.irradiance * panelArea); // W_in
        double vout = Math.max(0, ctx.dcVoltage * ctx.dcCurrent); // W_
out
        double eff = vout / vin;
        value = Math.max(0, Math.min(1.0, eff + (rnd.nextDouble() - 0.5
) * 0.01));
        ok = (value >= 0 && value <= 1.0);
        status = value > 0.2 ? "OK" : "LOW";
    }
}

class DCBusVoltageSensor extends SSensor {
    DCBusVoltageSensor() { super("DCBusVoltageSensor"); }
    @Override
    void update(SolarContext ctx) {
        value = Math.max(0, ctx.dcVoltage + (rnd.nextDouble() - 0.5) *
0.8);
        ok = (value >= 0 && value < 1000);
        status = ok ? "OK" : "OUT_OF_RANGE";
    }
}

class DCBusCurrentSensor extends SSensor {
    DCBusCurrentSensor() { super("DCBusCurrentSensor"); }
    @Override
    void update(SolarContext ctx) {
        value = Math.max(0, ctx.dcCurrent + (rnd.nextDouble() - 0.5) *
0.8);
        ok = (value >= 0 && value < 1000);
        status = ok ? "OK" : "OUT_OF_RANGE";
    }
}

```

```

/* ===== Controllers ===== */
class TiltController {
    private double minAngle = 0.0;    // degrees
    private double maxAngle = 90.0;   // degrees
    private double maxRate = 10.0;    // degrees per step
    private boolean enabled = true;
    double clampAngle(double desired) {
        return Math.max(minAngle, Math.min(maxAngle, desired));
    }
    double rateLimit(double current, double desired) {
        double d = desired - current;
        if (d > maxRate) d = maxRate;
        if (d < -maxRate) d = -maxRate;
        return current + d;
    }
    void setEnabled(boolean e) { enabled = e; }
    boolean isEnabled() { return enabled; }
    double getMinAngle() { return minAngle; }
    double getMaxAngle() { return maxAngle; }
    void setLimits(double minA, double maxA) {
        minAngle = Math.max(0, Math.min(minA, 90));
        maxAngle = Math.max(minAngle, Math.min(maxA, 90));
    }
    void setMaxRate(double r) { maxRate = Math.max(1.0, Math.min(30.0, r)); }
}

class MPPT {
    // Very small helper to suggest a power delta based on irradiance &
    // angle (toy model)
    double suggestPowerDelta(double irradiance, double angleDeg) {
        // toy: best angle ~ 35 deg; penalty grows with |angle-35|
        double penalty = Math.abs(angleDeg - 35.0) * 0.003;
        double base = irradiance * 0.5; // project-reference constant
        return base * (1.0 - penalty) * 0.01; // small incremental suggestion
    }
}

/* ===== Core Domain ===== */
class SmartSolarPanel {
    private final SDataLogger logger;
    private final SAlertSystem alert;
    // thresholds
    private double maxOptimalPower; // W
    private double minOptimalPower; // W
}

```

```

// state
private double currentPowerOutput; // W
private double panelTiltAngle;    // degrees
private boolean trackingEnabled = true;
// environment and derived values
private double irradiance = 600.0;    // W/m^2
private double cellTemperature = 45.0; // °C
private double ambientTemperature = 25.0; // °C
private double windSpeed = 2.0;      // m/s
private double dcVoltage = 200.0;    // V
private double dcCurrent = 1.5;      // A
private double cumulativeEnergyWh = 0.0; // Wh
// components
private final TiltController tilt = new TiltController();
private final MPPT mppt = new MPPT();
private final IrradianceSensor irSensor = new IrradianceSensor();
private final CellTempSensor cellTempSensor = new CellTempSensor();
private final EfficiencySensor effSensor = new EfficiencySensor();
private final DCBusVoltageSensor vSensor = new DCBusVoltageSensor()
;
private final DCBusCurrentSensor iSensor = new DCBusCurrentSensor()
;

// listeners
private final List<Runnable> listeners = new ArrayList<>();
SmartSolarPanel(double maxOptimalPower, double minOptimalPower, double initialPower, double initialAngle,
                SDataLogger logger, SAlertSystem alert) {
    this.maxOptimalPower = maxOptimalPower;
    this.minOptimalPower = minOptimalPower;
    this.currentPowerOutput = initialPower;
    this.panelTiltAngle = initialAngle;
    this.logger = logger;
    this.alert = alert;
    logger.log("SmartSolarPanel init: P=" + SStr.fmt2(currentPowerOutput) + "W, angle=" + SStr.fmt2(panelTiltAngle) +
               "°, min=" + SStr.fmt2(minOptimalPower) + "W, max=" + SStr.fmt2(maxOptimalPower) + "W");
}
void addListener(Runnable r) { listeners.add(r); }
private void emit() { for (Runnable r : listeners) try { r.run(); }
catch (Throwable ignored) {} }
SolarContext buildContext() {
    SolarContext c = new SolarContext();
    c.currentPowerOutput = currentPowerOutput;

```

```

        c.minOptimalPower = minOptimalPower;
        c.maxOptimalPower = maxOptimalPower;
        c.panelTiltAngle = panelTiltAngle;
        c.irradiance = irradiance;
        c.cellTemperature = cellTemperature;
        c.ambientTemperature = ambientTemperature;
        c.windSpeed = windSpeed;
        c.dcVoltage = dcVoltage;
        c.dcCurrent = dcCurrent;
        c.trackingEnabled = trackingEnabled;
        c.cumulativeEnergyWh = cumulativeEnergyWh;
        return c;
    }
    private void sampleSensors() {
        SolarContext ctx = buildContext();
        irSensor.update(ctx);
        cellTempSensor.update(ctx);
        effSensor.update(ctx);
        vSensor.update(ctx);
        iSensor.update(ctx);
        // Alerts on extremes
        if (!cellTempSensor.isOk() || cellTempSensor.getValue() > 85) {
            alert.push(SAlertSystem.Severity.WARNING, "High cell temperature: " + SStr.fmt2(cellTempSensor.getValue()) + "°C");
        }
        if (!irSensor.isOk()) {
            alert.push(SAlertSystem.Severity.INFO, "Irradiance sensor out-of-range.");
        }
    }
    /* =====
    * CORE FUNCTION: Keep original logic inside ONE function.
    * This function is intentionally the main branch-under-test:
    *   - currentPowerOutput += changeAmount
    *   - if (currentPowerOutput < minOptimalPower) panelTiltAngle +=
5
    *   - else if (currentPowerOutput > maxOptimalPower) panelTiltAngle
-= 5
    *   - else stay
    *   - Plus: clamp angle via controller and rate limit
    *   - Also: logging, sensors sampling, and notifying listeners
    * ===== */
    public synchronized void updatePowerOutput(double changeAmount) {
        // (1) Update power by delta (from user or MPI/MPPT)

```

```

        currentPowerOutput += changeAmount;
        // (2) Original one-
function branch logic: adjust tilt based on thresholds
        double desiredAngle = panelTiltAngle;
        if (currentPowerOutput < minOptimalPower) {
            desiredAngle = panelTiltAngle + 5.0; // increase tilt to ca
pture more
        } else if (currentPowerOutput > maxOptimalPower) {
            desiredAngle = panelTiltAngle - 5.0; // decrease tilt to av
oid clipping / optimize
        } // else: do nothing
        // (3) Apply physical constraints & rate limit by controller
//         if (tilt.isEnabled()) {
//             desiredAngle = tilt.clampAngle(desiredAngle);
//             panelTiltAngle = tilt.rateLimit(panelTiltAngle, desiredAngl
e);
//         }
        // (4) Update DC side roughly (toy model) after tilt change
        //         Assume DC voltage slowly rises with better alignment, cur
rent follows irradiance.
        double eff = Math.max(0.05, Math.min(0.25, 0.15 + (35.0 - Math.
abs(panelTiltAngle - 35.0)) * 0.002));
        dcVoltage = Math.max(50, 180 + (35.0 - Math.abs(panelTiltAngle
- 35.0)) * 2.0);
        dcCurrent = Math.max(0, (irradiance / 1000.0) * 10.0 * eff);
        // recompute power estimate (but keep currentPowerOutput as the
"state under test")
        double estimatedPower = dcVoltage * dcCurrent;
        // accumulate energy (Wh) per unit step assumption: ~1 second p
er update when sim/CLI calls often.
        cumulativeEnergyWh += estimatedPower / 3600.0;
        // (5) Log and sensors
        String msg = "Current Power Output: " + SStr.fmt2(currentPowerO
utput) + " W | Tilt: " + SStr.fmt2(panelTiltAngle) + "°";
        logger.log(msg);
        System.out.println(msg);
        sampleSensors();
        emit();
        // (6) Alerts around thresholds
        if (currentPowerOutput < minOptimalPower) {
            alert.push(SAlertSystem.Severity.INFO, "Power below minimum
optimal: " + SStr.fmt2(currentPowerOutput) + " < " + SStr.fmt2(minOptim
alPower));
        }

```

```

        if (currentPowerOutput > maxOptimalPower) {
            alert.push(SAlertSystem.Severity.INFO, "Power above maximum
optimal: " + SStr.fmt2(currentPowerOutput) + " > " + SStr.fmt2(maxOptim
alPower));
        }
    }
    /* ===== Simulation hooks ===== */
    synchronized void tickSeconds(double seconds) {
        // MPPT suggestion to nudge power (toy)
        double delta = mppt.suggestPowerDelta(irradiance, panelTiltAngl
e);
        // Use the SAME core function to keep coverage focus centralize
d
        updatePowerOutput(delta * seconds);
        // ambient & cell temperature dynamics (toy)
        cellTemperature += ((ambientTemperature + irradiance * 0.02) -
cellTemperature) * 0.01
            - windSpeed * 0.003;
    }
    synchronized void applyEnvironment(double irrDelta, double ambDelta
, double windDelta) {
        irradiance = Math.max(0, irradiance + irrDelta);
        ambientTemperature += ambDelta;
        windSpeed = Math.max(0, windSpeed + windDelta);
        // If extremely low irradiance, power tends to drop slowly if w
e don't push it up
        if (irradiance < 50) updatePowerOutput(-5.0);
        else if (irradiance > 1000) updatePowerOutput(+5.0);
        logger.log(String.format(Locale.US,
            "Env step: dIrr=%.3f W/m², dAmb=%.3f °C, dWind=%.3f m/s
| I=%.1f, Amb=%.1f, Wind=%.1f",
            irrDelta, ambDelta, windDelta, irradiance, ambientTempe
rature, windSpeed));
        emit();
    }
    /* ===== Maintenance/Config ===== */
    synchronized void calibratePower(double observed) {
        double old = currentPowerOutput;
        currentPowerOutput = Math.max(0, observed);
        logger.log("Calibration: power " + SStr.fmt2(old) + " -> " + SS
tr.fmt2(currentPowerOutput) + " W");
        sampleSensors();
        emit();
    }
}

```

```

    synchronized void setThresholds(double maxP, double minP) {
        double oldMax = maxOptimalPower, oldMin = minOptimalPower;
        maxOptimalPower = Math.max(minP + 1, maxP);
        minOptimalPower = Math.min(minP, maxOptimalPower - 1);
        logger.log("Thresholds changed: max " + SStr.fmt2(oldMax) + "->"
+ SStr.fmt2(maxOptimalPower) +
            ", min " + SStr.fmt2(oldMin) + "->" + SStr.fmt2(minOpti
malPower));
        sampleSensors();
        emit();
    }
    synchronized void setTrackingEnabled(boolean enabled) {
        trackingEnabled = enabled;
        tilt.setEnabled(enabled);
        logger.log("Tracking " + (enabled ? "ENABLED" : "DISABLED"));
    }
    synchronized void setTiltLimits(double minA, double maxA) {
        tilt.setLimits(minA, maxA);
        logger.log("Tilt limits set: [" + SStr.fmt2(tilt.getMinAngle())
+ "°, " + SStr.fmt2(tilt.getMaxAngle()) + "°]");
    }
    synchronized void setTiltMaxRate(double r) {
        tilt.setMaxRate(r);
        logger.log("Tilt max rate set: " + SStr.fmt2(r) + " °/step");
    }
    /* ===== Getters for GUI/tests ===== */
    double getCurrentPower() { return currentPowerOutput; }
    double getMinOptimalPower() { return minOptimalPower; }
    double getMaxOptimalPower() { return maxOptimalPower; }
    double getPanelTiltAngle() { return panelTiltAngle; }
    double getIrradiance() { return irradiance; }
    double getCellTemperature() { return cellTemperature; }
    double getAmbientTemperature() { return ambientTemperature; }
    double getWindSpeed() { return windSpeed; }
    double getDcVoltage() { return dcVoltage; }
    double getDcCurrent() { return dcCurrent; }
    double getCumulativeEnergyWh() { return cumulativeEnergyWh; }
    IrradianceSensor getIrradianceSensor() { return irSensor; }
    CellTempSensor getCellTempSensor() { return cellTempSensor; }
    EfficiencySensor getEfficiencySensor() { return effSensor; }
    DCBusVoltageSensor getVoltageSensor() { return vSensor; }
    DCBusCurrentSensor getCurrentSensor() { return iSensor; }
}

```

```

/* ===== Simulation Engine =====
= */
class SolarSimulationEngine implements Runnable {
    private final SmartSolarPanel solar;
    private final SDataLogger logger;
    private final AtomicBoolean running = new AtomicBoolean(false);
    private Thread thread;
    private volatile double irrDrift = +2.0;    // W/m^2 per tick
    private volatile double ambDrift = +0.02;   // °C per tick
    private volatile double windDrift = +0.01;  // m/s per tick
    private volatile double tickSeconds = 1.0;
    private volatile long sleepMillis = 250L;
    private volatile boolean randomize = true;
    private final Random rnd = new Random();
    SolarSimulationEngine(SmartSolarPanel solar, SDataLogger logger) {
        this.solar = solar;
        this.logger = logger;
    }
    void start() {
        if (running.get()) return;
        running.set(true);
        thread = new Thread(this, "SolarSimEngine");
        thread.setDaemon(true);
        thread.start();
        logger.log("Solar simulation started.");
    }
    void stop() {
        running.set(false);
        if (thread != null) {
            try { thread.join(2000); } catch (InterruptedException ignored) {}
        }
        logger.log("Solar simulation stopped.");
    }
    boolean isRunning() { return running.get(); }
    void setParams(double irrDrift, double ambDrift, double windDrift,
                  double tickSeconds, long delayMs, boolean randomize)
    {
        this.irrDrift = irrDrift;
        this.ambDrift = ambDrift;
        this.windDrift = windDrift;
        this.tickSeconds = Math.max(0.1, tickSeconds);
        this.sleepMillis = Math.max(20, delayMs);
        this.randomize = randomize;
    }
}

```

```

        logger.log(String.format(Locale.US,
            "Sim params updated: dIrr=%.3f, dAmb=%.3f, dWind=%.3f,
dt=%.2fs, delay=%dms, random=%s",
            irrDrift, ambDrift, windDrift, this.tickSeconds, this.sleepMillis, String.valueOf(this.randomize)));
    }
    @Override
    public void run() {
        while (running.get()) {
            double dI = irrDrift;
            double dA = ambDrift;
            double dW = windDrift;
            if (randomize) {
                dI += (rnd.nextDouble() - 0.5) * Math.max(5.0, Math.abs(
irrDrift) * 3.0);
                dA += (rnd.nextDouble() - 0.5) * Math.max(0.2, Math.abs(
ambDrift) * 2.0);
                dW = Math.max(0, dW + (rnd.nextDouble() - 0.5) * Math.m
ax(0.1, Math.abs(windDrift) * 3.0));
            }
            solar.applyEnvironment(dI, dA, dW);
            solar.tickSeconds(tickSeconds);
            try { Thread.sleep(sleepMillis); }
            catch (InterruptedException e) { break; }
        }
    }
}
/* ===== Replay (History) =====
*/
class SolarReplayEngine {
    private final SmartSolarPanel solar;
    private final SDataLogger logger;
    SolarReplayEngine(SmartSolarPanel solar, SDataLogger logger) {
        this.solar = solar;
        this.logger = logger;
    }
    void replay(File logFile, long delayMs) {
        if (logFile == null || !logFile.exists()) {
            System.err.println("[Replay] Log file not found.");
            return;
        }
        logger.log("Replay started from: " + logFile.getAbsolutePath())
;

```

```

        try (BufferedReader br = new BufferedReader(new InputStreamReader(
new FileInputStream(logFile), StandardCharsets.UTF_8))) {
            String line;
            while ((line = br.readLine()) != null) {
                if (!line.contains("Env step:")) continue;
                try {
                    int idx = line.indexOf("Env step:");
                    String seg = line.substring(idx);
                    // Env step: dIrr=..., dAmb=..., dWind=... | ...
                    String[] parts = seg.split("\\|")[0].split(",");
                    double dI = Double.parseDouble(parts[0].split("=")[
1].replace("W/m²", "").trim());
                    double dA = Double.parseDouble(parts[1].split("=")[
1].replace("°C", "").trim());
                    double dW = Double.parseDouble(parts[2].split("=")[
1].replace("m/s", "").trim());
                    solar.applyEnvironment(dI, dA, dW);
                    solar.tickSeconds(1.0);
                    Thread.sleep(delayMs);
                } catch (Throwable ignored) {}
            }
        } catch (IOException e) {
            System.err.println("[Replay] Error: " + e.getMessage());
        }
        logger.log("Replay finished.");
    }
}

/* ===== GUI ===== */
class SolarGUI extends JFrame {
    private final SmartSolarPanel solar;
    private final SDataLogger logger;
    private final SAlertSystem alerts;
    private final SolarSimulationEngine engine;
    private final SolarReplayEngine replay;
    private JLabel lblP, lblMin, lblMax, lblAngle, lblIrr, lblCellT, lb
1AmbT, lblWind, lblV, lblI, lblEWh;
    private JTable tblSensors;
    private DefaultTableModel tblModel;
    private JTextArea txtLog, txtAlerts;
    private JSpinner spMin, spMax, spMinA, spMaxA, spRate;
    private JCheckBox cbTrack;
    private JSpinner spdIrr, spdAmb, spdWind, spDt, spDelay;
    private JCheckBox cbRandomize;

```

```

    private JButton btnStart, btnStop, btnReplay, btnCalib, btnAdd, btn
Sub;
    private javax.swing.Timer uiTimer;
    SolarGUI(SmartSolarPanel solar, SDataLogger logger, SAlertSystem al
erts,
        SolarSimulationEngine engine, SolarReplayEngine replay) {
        super("SmartSolar - Tilt & Power Optimization");
        this.solar = solar;
        this.logger = logger;
        this.alerts = alerts;
        this.engine = engine;
        this.replay = replay;
        setDefaultCloseOperation(WindowConstants.EXIT_ON_CLOSE);
        buildUi();
        setSize(1200, 780);
        setLocationRelativeTo(null);
        uiTimer = new javax.swing.Timer(500, e -> refreshUi());
        uiTimer.start();
        solar.addListener(this::refreshUi);
    }
    private JPanel titled(String title, JComponent inner) {
        JPanel p = new JPanel(new BorderLayout());
        p.setBorder(BorderFactory.createTitledBorder(title));
        p.add(inner, BorderLayout.CENTER);
        return p;
    }
    private void buildUi() {
        JPanel root = new JPanel(new BorderLayout(10,10));
        root.setBorder(new EmptyBorder(10,10,10,10));
        setContentPane(root);
        /* LEFT: Status & Controls */
        JPanel left = new JPanel();
        left.setLayout(new BoxLayout(left, BoxLayout.Y_AXIS));
        JPanel status = new JPanel(new GridLayout(0,2,6,6));
        lblP = new JLabel("-");
        lblMin = new JLabel("-");
        lblMax = new JLabel("-");
        lblAngle = new JLabel("-");
        lblIrr = new JLabel("-");
        lblCellT = new JLabel("-");
        lblAmbT = new JLabel("-");
        lblWind = new JLabel("-");
        lblV = new JLabel("-");
        lblI = new JLabel("-");

```

```

        lblEWh = new JLabel("-");
        status.add(new JLabel("Power (W):"));        status.add(lblP);
        status.add(new JLabel("MinOpt (W):"));        status.add(lblMin);
        status.add(new JLabel("MaxOpt (W):"));        status.add(lblMax);
        status.add(new JLabel("Tilt (°):"));        status.add(lblAngle)
;
        status.add(new JLabel("Irrad (W/m²):"));        status.add(lblIrr);
        status.add(new JLabel("CellT (°C):"));        status.add(lblCellT)
;
        status.add(new JLabel("Ambient (°C):"));        status.add(lblAmbT);
        status.add(new JLabel("Wind (m/s):"));        status.add(lblWind);
        status.add(new JLabel("Vdc (V):"));        status.add(lblV);
        status.add(new JLabel("Idc (A):"));        status.add(lblI);
        status.add(new JLabel("Energy (Wh):"));        status.add(lblEWh);
        left.add(titled("Live Status", status));
        JPanel th = new JPanel(new GridLayout(0,2,6,6));
        spMin = new JSpinner(new SpinnerNumberModel(200.0, 0.0, 2000.0,
5.0));
        spMax = new JSpinner(new SpinnerNumberModel(500.0, 0.0, 5000.0,
5.0));
        cbTrack = new JCheckBox("Tracking Enabled", true);
        JButton btnApply = new JButton("Apply Thresholds");
        btnApply.addActionListener(e -> {
            double mn = ((Number)spMin.getValue()).doubleValue();
            double mx = ((Number)spMax.getValue()).doubleValue();
            solar.setThresholds(mx, mn);
            solar.setTrackingEnabled(cbTrack.isSelected());
        });
        th.add(new JLabel("Min optimal (W)")); th.add(spMin);
        th.add(new JLabel("Max optimal (W)")); th.add(spMax);
        th.add(new JLabel(" "));        th.add(cbTrack);
        th.add(new JLabel(" "));        th.add(btnApply);
        left.add(Box.createVerticalStrut(6));
        left.add(titled("Thresholds", th));
        JPanel tiltCfg = new JPanel(new GridLayout(0,2,6,6));
        spMinA = new JSpinner(new SpinnerNumberModel(0.0, 0.0, 90.0, 1.
0));
        spMaxA = new JSpinner(new SpinnerNumberModel(90.0, 0.0, 90.0, 1
.0));
        spRate = new JSpinner(new SpinnerNumberModel(10.0, 1.0, 30.0, 1
.0));
        JButton btnTiltCfg = new JButton("Apply Tilt Limits/Rate");
        btnTiltCfg.addActionListener(e -> {

```

```

        solar.setTiltLimits(((Number)spMinA.getValue()).doubleValue
()),
                                ((Number)spMaxA.getValue()).doubleValue
());
        solar.setTiltMaxRate(((Number)spRate.getValue()).doubleValu
e());
    });
    tiltCfg.add(new JLabel("Tilt min (°)")); tiltCfg.add(spMinA);
    tiltCfg.add(new JLabel("Tilt max (°)")); tiltCfg.add(spMaxA);
    tiltCfg.add(new JLabel("Max rate (°/step)")); tiltCfg.add(spRate);
    tiltCfg.add(new JLabel(" ")); tiltCfg.add(btnTiltCfg);
    left.add(Box.createVerticalStrut(6));
    left.add(titled("Tilt Controller", tiltCfg));
    JPanel ops = new JPanel(new GridLayout(0,2,6,6));
    btnAdd = new JButton("Manual +Power +50W");
    btnSub = new JButton("Manual -Power -50W");
    btnCalib = new JButton("Calibrate Power");
    btnAdd.addActionListener(e -> solar.updatePowerOutput(+50.0));
    btnSub.addActionListener(e -> solar.updatePowerOutput(-50.0));
    btnCalib.addActionListener(e -> {
        String s = JOptionPane.showInputDialog(this, "Observed power (W):", SStr.fmt2(solar.getCurrentPower()));
        if (s == null) return;
        try { solar.calibratePower(Double.parseDouble(s)); }
        catch (NumberFormatException ex) { JOptionPane.showMessageDialog(this, "Invalid number."); }
    });
    ops.add(btnAdd); ops.add(btnSub);
    ops.add(btnCalib); ops.add(new JLabel(" "));
    left.add(Box.createVerticalStrut(6));
    left.add(titled("Operations", ops));
    JPanel sim = new JPanel(new GridLayout(0,2,6,6));
    spdIrr = new JSpinner(new SpinnerNumberModel(+2.0, -100.0, 100.0, 1.0));
    spdAmb = new JSpinner(new SpinnerNumberModel(+0.02, -5.0, 5.0, 0.01));
    spdWind= new JSpinner(new SpinnerNumberModel(+0.01, 0.0, 10.0, 0.01));
    spDt    = new JSpinner(new SpinnerNumberModel(1.0, 0.1, 10.0, 0.1));
    spDelay= new JSpinner(new SpinnerNumberModel(250, 20, 2000, 10));
    cbRandomize = new JCheckBox("Randomize", true);

```

```

        btnStart = new JButton("Start Simulation");
        btnStop = new JButton("Stop Simulation");
        btnReplay = new JButton("Replay from Log");
        btnStart.addActionListener(e -> engine.setParams(
            ((Number)spdIrr.getValue()).doubleValue(),
            ((Number)spdAmb.getValue()).doubleValue(),
            ((Number)spdWind.getValue()).doubleValue(),
            ((Number)spDt.getValue()).doubleValue(),
            ((Number)spDelay.getValue()).longValue(),
            cbRandomize.isSelected()
        ));
        btnStart.addActionListener(e -> engine.start());
        btnStop.addActionListener(e -> engine.stop());
        btnReplay.addActionListener(e -> new Thread(() -> replay.replay
(logger.getFile(), 60), "Replay").start());
        sim.add(new JLabel("ΔIrr per tick (W/m²)")); sim.add(spdIrr);
        sim.add(new JLabel("ΔAmb per tick (°C)")); sim.add(spdAmb);
        sim.add(new JLabel("ΔWind per tick (m/s)")); sim.add(spdWind);
        sim.add(new JLabel("dt (s)")); sim.add(spDt);
        sim.add(new JLabel("Delay (ms)")); sim.add(spDelay);
        sim.add(new JLabel(" ")); sim.add(cbRandomize
);

        sim.add(btnStart); sim.add(btnStop);
        sim.add(new JLabel(" ")); sim.add(btnReplay);
        left.add(Box.createVerticalStrut(6));
        left.add(titled("Simulation", sim));
        getContentPane().add(left, BorderLayout.WEST);
        /* CENTER: Sensors + Alerts */
        JPanel center = new JPanel(new BorderLayout(6,6));
        tblModel = new DefaultTableModel(new String[]{"Sensor", "Value",
"OK", "Status"}, 0) {
            @Override public boolean isCellEditable(int row, int column
) { return false; }
        };
        tblSensors = new JTable(tblModel);
        JScrollPane spTbl = new JScrollPane(tblSensors);
        center.add(titled("Sensors", spTbl), BorderLayout.CENTER);
        txtAlerts = new JTextArea(8, 60);
        txtAlerts.setEditable(false);
        center.add(titled("Recent Alerts", new JScrollPane(txtAlerts)),
BorderLayout.SOUTH);
        getContentPane().add(center, BorderLayout.CENTER);
        /* RIGHT: Log tail */
        txtLog = new JTextArea();

```

```

        txtLog.setEditable(false);
        JScrollPane spLog = new JScrollPane(txtLog);
        getContentPane().add(titled("Log (tail)", spLog), BorderLayout.
EAST);
    }
    private void refreshUi() {
        lblP.setText(SStr.fmt2(solar.getCurrentPower()));
        lblMin.setText(SStr.fmt2(solar.getMinOptimalPower()));
        lblMax.setText(SStr.fmt2(solar.getMaxOptimalPower()));
        lblAngle.setText(SStr.fmt2(solar.getPanelTiltAngle()));
        lblIrr.setText(SStr.fmt2(solar.getIrradiance()));
        lblCellT.setText(SStr.fmt2(solar.getCellTemperature()));
        lblAmbT.setText(SStr.fmt2(solar.getAmbientTemperature()));
        lblWind.setText(SStr.fmt2(solar.getWindSpeed()));
        lblV.setText(SStr.fmt2(solar.getDcVoltage()));
        lblI.setText(SStr.fmt2(solar.getDcCurrent()));
        lblEWh.setText(SStr.fmt2(solar.getCumulativeEnergyWh()));
        // sensors table
        if (tblModel != null) {
            tblModel.setRowCount(0);
            tblModel.addRow(new Object[]{
                solar.getIrradianceSensor().getName(),
                SStr.fmt2(solar.getIrradianceSensor().getValue()),
                solar.getIrradianceSensor().isOk(), solar.getIrradi
anceSensor().getStatus()
            });
            tblModel.addRow(new Object[]{
                solar.getCellTempSensor().getName(),
                SStr.fmt2(solar.getCellTempSensor().getValue()),
                solar.getCellTempSensor().isOk(), solar.getCellTemp
Sensor().getStatus()
            });
            tblModel.addRow(new Object[]{
                solar.getEfficiencySensor().getName(),
                SStr.fmt3(solar.getEfficiencySensor().getValue()),
                solar.getEfficiencySensor().isOk(), solar.getEffici
encySensor().getStatus()
            });
            tblModel.addRow(new Object[]{
                solar.getVoltageSensor().getName(),
                SStr.fmt2(solar.getVoltageSensor().getValue()),
                solar.getVoltageSensor().isOk(), solar.getVoltageSe
nsor().getStatus()
            });

```

```

        tblModel.addRow(new Object[]{
            solar.getCurrentSensor().getName(),
            SStr.fmt2(solar.getCurrentSensor().getValue()),
            solar.getCurrentSensor().isOk(), solar.getCurrentSe
nsor().getStatus()
        });
    }
    // alerts
    if (txtAlerts != null) {
        StringBuilder sb = new StringBuilder();
        for (SAlertSystem.Alert a : alerts.recent(18)) sb.append(a)
.append("\n");
        txtAlerts.setText(sb.toString());
        txtAlerts.setCaretPosition(txtAlerts.getDocument().getLengt
h());
    }
    // log tail
    if (txtLog != null) {
        List<String> lines = logger.recent(220);
        StringBuilder sb = new StringBuilder();
        for (String s : lines) sb.append(s).append("\n");
        txtLog.setText(sb.toString());
        txtLog.setCaretPosition(txtLog.getDocument().getLength());
    }
}
}
/* ===== CLI Harness ===== */
class SolarCLIHarness {
    private final SmartSolarPanel solar;
    SolarCLIHarness(SmartSolarPanel solar) { this.solar = solar; }
    void runAll() {
        System.out.println("==== CLI Demo (VT) ====");
        vt();
        System.out.println("==== CLI Demo (FT) ====");
        ft();
        System.out.println("==== CLI Demo (Z3 mimic) ====");
        z3();
        System.out.println("==== CLI Demo (UVT) ====");
        uvt();
    }
    // VT: use the given sequence to trigger <min, between, >max transi
tions via the core function.
    void vt() {

```

```

        solar.updatePowerOutput(-
150.0); // 300 -> 150 (< min=200) => +5°
        solar.updatePowerOutput(100.0); // 150 -> 250 (between) => stay
        solar.updatePowerOutput(300.0); // 250 -> 550 (> max=500) => -
5°
        solar.updatePowerOutput(-50.0); // 550 -> 500 (edge) => stay
    }
    // FT: extreme numeric stress
    void ft() {
        solar.updatePowerOutput(-4.9E-324);
        solar.updatePowerOutput(0.0);
        solar.updatePowerOutput(-1.7976931348623157E308);
        solar.updatePowerOutput(2.2250738585072014E-308);
    }
    // Z3-like: mirror VT for deterministic coverage
    void z3() {
        solar.updatePowerOutput(-150.0);
        solar.updatePowerOutput(100.0);
        solar.updatePowerOutput(300.0);
        solar.updatePowerOutput(-50.0);
    }
    // UVT: mirror VT again
    void uvt() {
        solar.updatePowerOutput(-150.0);
        solar.updatePowerOutput(100.0);
        solar.updatePowerOutput(300.0);
        solar.updatePowerOutput(-50.0);
    }
}
/* ===== Main ===== */
public class c129_SmartSolarSystemEx {
    public static void main(String[] args) {
        // boolean forceCli = Arrays.asList(args).contains("--cli");
        boolean forceCli = true;
        // Logger & Alerts
        SDataLogger logger = new SDataLogger("solar_log.txt", 900);
        SAlertSystem alertSystem = new SAlertSystem(logger);
        // Core panel with user's parameters
        SmartSolarPanel solar = new SmartSolarPanel(500.0, 200.0, 300.0
, 30.0, logger, alertSystem);
        // Engines
        SolarSimulationEngine engine = new SolarSimulationEngine(solar,
logger);

```

```

        SolarReplayEngine replay = new SolarReplayEngine(solar, logger)
;
        // ===== CLI path (sequential 4 groups; comment 3 lines to run
single group) =====
        if (forceCli || headless()) {
            logger.log("Running in CLI mode.");
            SolarCLIHarness harness = new SolarCLIHarness(solar);
            harness.vt(); // ← keep to run VT
//            harness.ft(); // ← comment out to skip FT
//            harness.z3(); // ← comment out to skip Z3
//            harness.uvt(); // ← comment out to skip UVT
            // short sim to exercise sensors
            for (int i = 0; i < 20; i++) {
                solar.applyEnvironment(+2.0, +0.05, +0.02);
                solar.tickSeconds(1.0);
                try { Thread.sleep(100L); } catch (Exception ignored) {
}
                    }
                logger.log("CLI demo finished.");
                System.out.println("CLI demo finished. Log written to: " +
logger.getFile().getAbsolutePath());
                return;
            }
            // ===== GUI path =====
            logger.log("Running in GUI mode.");
            SwingUtilities.invokeLater(() -> {
                SolarGUI gui = new SolarGUI(solar, logger, alertSystem, eng
ine, replay);
                gui.setVisible(true);
            });
        }
        private static boolean headless() {
            try { return GraphicsEnvironment.isHeadless(); }
            catch (Throwable t) { return true; }
        }
    }
}

```

- Coverage report

1. Test case number: 4
2. V-Method testing branch coverage: 100%

Element	Coverage	Covered Branches	Missed Branches	Total Branches
▼ c129_SmartSolarSystemEx.java	34.9 %	37	69	106
> SAlertSystem	0.0 %	0	2	2
> SolarReplayEngine	0.0 %	0	8	8
> SolarSimulationEngine	0.0 %	0	8	8
> TiltController	0.0 %	0	4	4
> SDataLogger	25.0 %	2	6	8
> c129_SmartSolarSystemEx	50.0 %	3	3	6
> CellTempSensor	50.0 %	4	4	8
> DCBusCurrentSensor	50.0 %	3	3	6
> DCBusVoltageSensor	50.0 %	3	3	6
> EfficiencySensor	50.0 %	3	3	6
> IrradianceSensor	50.0 %	5	5	10
▼ SmartSolarPanel	63.6 %	14	8	22
▶ applyEnvironment(double, double, d	50.0 %	2	2	4
▶ emit()	50.0 %	1	1	2
▶ sampleSensors()	50.0 %	3	3	6
▶ updatePowerOutput(double)	100.0 %	8	0	8
▶ SmartSolarPanel(double, double, doi		0	0	0

3. Vibration testing instruction coverage: 100%

Element	Coverage	Covered Instructions	Missed Instructions	Total Instructions
▼ c129_SmartSolarSystemEx.java	31.9 %	1,070	2,286	3,356
> SolarContext	5.3 %	3	54	57
> SolarReplayEngine	6.9 %	9	122	131
> SolarSimulationEngine	16.0 %	38	199	237
> TiltController	17.4 %	15	71	86
> SolarCLIHarness	24.2 %	23	72	95
> SStr	54.5 %	12	10	22
> SAlertSystem	55.9 %	33	26	59
> SDataLogger	57.9 %	70	51	121
> c129_SmartSolarSystemEx	68.2 %	75	35	110
▼ SmartSolarPanel	70.4 %	454	191	645
▶ emit()	52.9 %	9	8	17
▶ sampleSensors()	73.1 %	38	14	52
▶ applyEnvironment(double, double, d	91.6 %	76	7	83
▶ SmartSolarPanel(double, double, doi	100.0 %	98	0	98
▶ buildContext()	100.0 %	54	0	54
▶ tickSeconds(double)	100.0 %	36	0	36
▶ updatePowerOutput(double)	100.0 %	143	0	143

4. Fuzz testing branch coverage: 75%

Element	Coverage	Covered Branches	Missed Branches	Total Branches
▼ c129_SmartSolarSystemEx.java	33.0 %	35	71	106
> SAlertSystem	0.0 %	0	2	2
> SolarReplayEngine	0.0 %	0	8	8
> SolarSimulationEngine	0.0 %	0	8	8
> TiltController	0.0 %	0	4	4
> SDataLogger	25.0 %	2	6	8
> c129_SmartSolarSystemEx	50.0 %	3	3	6
> CellTempSensor	50.0 %	4	4	8
> DCBusCurrentSensor	50.0 %	3	3	6
> DCBusVoltageSensor	50.0 %	3	3	6
> EfficiencySensor	50.0 %	3	3	6
> IrradianceSensor	50.0 %	5	5	10
▼ SmartSolarPanel	54.5 %	12	10	22
▶ applyEnvironment(double, double, d	50.0 %	2	2	4
▶ emit()	50.0 %	1	1	2
▶ sampleSensors()	50.0 %	3	3	6
▶ updatePowerOutput(double)	75.0 %	6	2	8
▶ SmartSolarPanel(double, double, doi		0	0	0
▶ buildContext()		0	0	0
▶ tickSeconds(double)		0	0	0

5. Fuzz testing instruction coverage: 88.8%

Element	Coverage	Covered Instructions	Missed Instructions	Total Instructions
▼ c129_SmartSolarSystemEx.java	31.4 %	1,054	2,302	3,356
> SolarContext	5.3 %	3	54	57
> SolarReplayEngine	6.9 %	9	122	131
> SolarSimulationEngine	16.0 %	38	199	237
> TiltController	17.4 %	15	71	86
> SolarCLIHarness	24.2 %	23	72	95
> SStr	54.5 %	12	10	22
> SAlertSystem	55.9 %	33	26	59
> SDataLogger	57.9 %	70	51	121
▼ SmartSolarPanel	67.9 %	438	207	645
emit()	52.9 %	9	8	17
sampleSensors()	73.1 %	38	14	52
updatePowerOutput(double)	88.8 %	127	16	143
applyEnvironment(double, double, d	91.6 %	76	7	83
SmartSolarPanel(double, double, doi	100.0 %	98	0	98
buildContext()	100.0 %	54	0	54
tickSeconds(double)	100.0 %	36	0	36
> c129_SmartSolarSystemEx	68.2 %	75	35	110
> SSensor	79.3 %	23	6	29
> IrradianceSensor	89.3 %	50	6	56

6. Z3 branch coverage: 100%

Element	Coverage	Covered Branches	Missed Branches	Total Branches
▼ c129_SmartSolarSystemEx.java	34.9 %	37	69	106
> SAlertSystem	0.0 %	0	2	2
> SolarReplayEngine	0.0 %	0	8	8
> SolarSimulationEngine	0.0 %	0	8	8
> TiltController	0.0 %	0	4	4
> SDataLogger	25.0 %	2	6	8
> c129_SmartSolarSystemEx	50.0 %	3	3	6
> CellTempSensor	50.0 %	4	4	8
> DCBusCurrentSensor	50.0 %	3	3	6
> DCBusVoltageSensor	50.0 %	3	3	6
> EfficiencySensor	50.0 %	3	3	6
> IrradianceSensor	50.0 %	5	5	10
▼ SmartSolarPanel	63.6 %	14	8	22
applyEnvironment(double, double, d	50.0 %	2	2	4
emit()	50.0 %	1	1	2
sampleSensors()	50.0 %	3	3	6
updatePowerOutput(double)	100.0 %	8	0	8
SmartSolarPanel(double, double, doi		0	0	0

7. Z3 instruction coverage: 100%

Element	Coverage	Covered Instructions	Missed Instructions	Total Instructions
▼ c129_SmartSolarSystemEx.java	31.9 %	1,070	2,286	3,356
> SolarContext	5.3 %	3	54	57
> SolarReplayEngine	6.9 %	9	122	131
> SolarSimulationEngine	16.0 %	38	199	237
> TiltController	17.4 %	15	71	86
> SolarCLIHarness	24.2 %	23	72	95
> SStr	54.5 %	12	10	22
> SAlertSystem	55.9 %	33	26	59
> SDataLogger	57.9 %	70	51	121
> c129_SmartSolarSystemEx	68.2 %	75	35	110
▼ SmartSolarPanel	70.4 %	454	191	645
emit()	52.9 %	9	8	17
sampleSensors()	73.1 %	38	14	52
applyEnvironment(double, double, d	91.6 %	76	7	83
SmartSolarPanel(double, double, doi	100.0 %	98	0	98
buildContext()	100.0 %	54	0	54
tickSeconds(double)	100.0 %	36	0	36
updatePowerOutput(double)	100.0 %	143	0	143
> SSensor	79.3 %	23	6	29
> IrradianceSensor	89.3 %	50	6	56

8. Unimproved V-method branch coverage: 100%

Element	Coverage	Covered Branches	Missed Branches	Total Branches
▼ c129_SmartSolarSystemEx.java	34.9 %	37	69	106
> SAAlertSystem	0.0 %	0	2	2
> SolarReplayEngine	0.0 %	0	8	8
> SolarSimulationEngine	0.0 %	0	8	8
> TiltController	0.0 %	0	4	4
> SDataLogger	25.0 %	2	6	8
> c129_SmartSolarSystemEx	50.0 %	3	3	6
> CellTempSensor	50.0 %	4	4	8
> DCBusCurrentSensor	50.0 %	3	3	6
> DCBusVoltageSensor	50.0 %	3	3	6
> EfficiencySensor	50.0 %	3	3	6
> IrradianceSensor	50.0 %	5	5	10
▼ SmartSolarPanel	63.6 %	14	8	22
applyEnvironment(double, double, d	50.0 %	2	2	4
emit()	50.0 %	1	1	2
sampleSensors()	50.0 %	3	3	6
updatePowerOutput(double)	100.0 %	8	0	8
SmartSolarPanel(double, double, doi		0	0	0

9. Unimproved V-method instruction coverage: 100%

Element	Coverage	Covered Instructions	Missed Instructions	Total Instructions
▼ c129_SmartSolarSystemEx.java	31.9 %	1,070	2,286	3,356
> SolarContext	5.3 %	3	54	57
> SolarReplayEngine	6.9 %	9	122	131
> SolarSimulationEngine	16.0 %	38	199	237
> TiltController	17.4 %	15	71	86
> SolarCLIHarness	24.2 %	23	72	95
> SStr	54.5 %	12	10	22
> SAAlertSystem	55.9 %	33	26	59
> SDataLogger	57.9 %	70	51	121
> c129_SmartSolarSystemEx	68.2 %	75	35	110
▼ SmartSolarPanel	70.4 %	454	191	645
emit()	52.9 %	9	8	17
sampleSensors()	73.1 %	38	14	52
applyEnvironment(double, double, d	91.6 %	76	7	83
SmartSolarPanel(double, double, doi	100.0 %	98	0	98
buildContext()	100.0 %	54	0	54
tickSeconds(double)	100.0 %	36	0	36
updatePowerOutput(double)	100.0 %	143	0	143